

LMSR for Binary Markets

Sticking to binary markets is a good call. It's the version that maps cleanly to hardware, and multi-outcome support can be a future extension. Here's LMSR rebuilt from scratch, with each formula derived from a requirement rather than stated.

1. The setup

A binary market on some event. A **YES share** pays \$1 if the event happens, \$0 otherwise. A **NO share** pays the reverse.

We want an automated market maker (MM) that:

1. will **always** trade, so there's never a missing counterparty;
2. quotes prices that make sense as probabilities;
3. can't be drained: its worst-case loss is a known constant.

The MM keeps track of how many shares of each it has sold:

$$q = (q_Y, q_N)$$

These are its **liabilities**. If YES happens it owes q_Y dollars; if NO happens it owes q_N .

2. Why use a cost function?

Let all the money the MM has ever collected be some function of what it has sold, $C(q_Y, q_N)$. A trade that moves the state from q to q' costs the difference:

$$\text{cost of trade} = C(q') - C(q)$$

This design choice alone buys three properties:

No gaming by order-splitting. Buying 100 shares at once or as 100 single shares costs the same, because the sum telescopes:

$$[C(q_1) - C(q_0)] + [C(q_2) - C(q_1)] + \dots = C(q_{\text{final}}) - C(q_0)$$

No free money from round trips. Buy and then sell back the same shares, and you end at the same q , so your net cost is exactly 0.

Prices come out as derivatives. Buying a tiny amount ε of YES costs

$$C(q_Y + \varepsilon, q_N) - C(q_Y, q_N) \approx \frac{\partial C}{\partial q_Y} \varepsilon$$

so the price per share is

$$p_Y = \frac{\partial C}{\partial q_Y}, \quad p_N = \frac{\partial C}{\partial q_N}$$

All that remains is choosing C . The requirements decide it for us.

3. What C must satisfy

Requirement A: YES + NO must cost exactly \$1. One YES plus one NO is a guaranteed \$1 payout, whatever happens. If the pair cost less than \$1, anyone could buy it for a riskless profit. If it cost more, the MM would be overcharging for certainty. So adding t to both coordinates must raise C by exactly t :

$$C(q_Y + t, q_N + t) = C(q_Y, q_N) + t$$

Differentiate both sides with respect to t at $t = 0$:

$$\frac{\partial C}{\partial q_Y} + \frac{\partial C}{\partial q_N} = 1 \implies p_Y + p_N = 1$$

The prices sum to 1 because otherwise there'd be a riskless trade, not because we imposed it.

Requirement B: buying should raise the price. The more YES gets bought, the more expensive the next YES share should be. So $\partial p_Y / \partial q_Y > 0$, which means C must be **convex**.

Requirement C: bounded loss. This one picks out the specific formula.

4. The key idea: charge traders a smooth max of your liabilities

At settlement, the MM pays out either q_Y or q_N . In the worst case it pays $\max(q_Y, q_N)$. The money it has collected is $C(q) - C(0)$, so:

$$\text{worst-case loss} = \max(q_Y, q_N) - [C(q) - C(0)]$$

To bound this by a constant, the natural choice is to make C track the max:

$$C(q) \approx \max(q_Y, q_N)$$

The actual max fails as a cost function: it isn't differentiable when $q_Y = q_N$, and its "price" would jump between 0 and 1. That isn't a probability anyone wants to quote. We need a **smooth** max.

The standard smooth max is **log-sum-exp**:

$$C(q_Y, q_N) = b \ln \left(e^{q_Y/b} + e^{q_N/b} \right)$$

That is LMSR's cost function. Here's why it's a smooth max.

Say q_Y is the larger one. Factor out $e^{q_Y/b}$:

$$C = b \ln \left(e^{q_Y/b} (1 + e^{(q_N - q_Y)/b}) \right) = q_Y + b \ln(1 + e^{-(q_Y - q_N)/b})$$

The correction term $b \ln(1 + e^{-\text{gap}/b})$:

- is at most $b \ln 2$, reached when the two are tied (gap = 0);

- shrinks toward 0 as the gap grows, so C approaches the plain max.

So we get the sandwich

$$\max(q_Y, q_N) \leq C(q) \leq \max(q_Y, q_N) + b \ln 2$$

b is the smoothing scale. Large b means a gradual, rounded max. Small b means a sharp, nearly true max.

Check it against the requirements:

- **A:** adding t to both inside the exponentials multiplies the sum by $e^{t/b}$, and $b \ln(e^{t/b}) = t$. ✓
- **B:** log-sum-exp is convex. ✓
- **C:** covered in §7. ✓

5. The prices: softmax, and what it means

Differentiate:

$$p_Y = \frac{\partial C}{\partial q_Y} = b \cdot \frac{\frac{1}{b} e^{q_Y/b}}{e^{q_Y/b} + e^{q_N/b}} = \frac{e^{q_Y/b}}{e^{q_Y/b} + e^{q_N/b}}$$

This is the **softmax**. Intuitively, the derivative of a smooth max asks how much each input currently counts as the max. If $q_Y \gg q_N$, C moves almost one-for-one with q_Y , so $p_Y \approx 1$. If they're tied, each matters half as much, so $p_Y = \frac{1}{2}$.

Dividing the top and bottom by $e^{q_Y/b}$ shows that only the **imbalance** matters:

$$p_Y = \frac{1}{1 + e^{-(q_Y - q_N)/b}} = \sigma\left(\frac{q_Y - q_N}{b}\right)$$

where σ is the logistic sigmoid. Now invert it. The **log-odds** of the price are

$$\ln \frac{p_Y}{1 - p_Y} = \frac{q_Y - q_N}{b}$$

This is the cleanest way to think about LMSR: it's linear in log-odds. Every b net YES shares bought raises the market's log-odds by exactly 1. Some consequences:

Net YES shares bought	Log-odds	Price
0	0	0.50
b	1	0.73
$2b$	2	0.88
$3b$	3	0.95

Moving the price from p to p' takes

$$\Delta = b \left[\ln \frac{p'}{1 - p'} - \ln \frac{p}{1 - p} \right] \text{ net shares}$$

so going from 50¢ to 60¢ takes $0.41b$ shares, while going from 90¢ to 99¢ takes $2.4b$. **Prices get sticky near certainty**, because log-odds stretch out near 0 and 1. That's the right behavior: confidence should get more expensive the closer it gets to certain.

6. Why traders report their true beliefs

Take a trader who moves the state from q to q' , changing the price from p to p' . Compute their profit in the world where outcome k happens (k is YES or NO).

First, a useful identity. From the cost function:

$$C(q) - q_k = b \ln(e^{q_Y/b} + e^{q_N/b}) - b \ln e^{q_k/b} = -b \ln \frac{e^{q_k/b}}{e^{q_Y/b} + e^{q_N/b}} = -b \ln p_k(q)$$

The trader's profit is their payout minus their cost:

$$\text{profit}_k = (q'_k - q_k) - [C(q') - C(q)] = [C(q) - q_k] - [C(q') - q'_k]$$

$$\text{profit}_k = b \ln p'_k - b \ln p_k$$

Your profit is how much you improved the market's log-probability on the outcome that actually happened. Push the price toward the truth and you're paid. Push it away and you pay.

Now say your honest belief is that YES has probability r . Your expected profit from moving the price to p' is

$$\mathbb{E}[\text{profit}] = b[r \ln p'_Y + (1 - r) \ln(1 - p'_Y)] - \text{const}$$

Set the derivative with respect to p'_Y to zero:

$$\frac{r}{p'_Y} - \frac{1 - r}{1 - p'_Y} = 0 \implies p'_Y = r$$

Your best move is to trade until the price equals your true belief. Stopping early leaves profit on the table, and going past it loses money in expectation. This is the **log scoring rule**, a “proper” scoring rule, and it's where LMSR gets its name. It's also why LMSR prices aggregate information: everyone's best strategy is to push the price to what they honestly believe.

7. Bounded loss, and what the MM is paying for

From the market maker's side, starting at $q = (0, 0)$, its profit if outcome k happens is collected minus paid:

$$\text{MM profit}_k = [C(q) - C(0)] - q_k = -b \ln p_k(q) - b \ln 2 = -b \ln(2 p_k)$$

using $C(0) = b \ln 2$. Read off the cases:

- Final price on the true outcome is $p_k = \frac{1}{2}$: profit is 0.
- The market ends **wrong** ($p_k \rightarrow 0$): profit is **positive**, because traders paid to push the price the wrong way.
- The market ends **right** ($p_k \rightarrow 1$): profit approaches $-b \ln 2$, the worst case.

$$\text{worst-case loss} = b \ln 2$$

The MM loses the most exactly when the market discovers the truth. **It's buying information.** $b \ln 2$ is the subsidy it pays for a market that converges to the right answer, and it is capped no matter how anyone trades. With $b = 100$, the maximum loss is about \$69.31.

8. Worked example

$b = 100$, starting at $q = (0, 0)$:

$$C(0, 0) = 100 \ln(1 + 1) = 69.31, \quad p_Y = 0.50$$

You buy 50 YES:

$$C(50, 0) = 100 \ln(e^{0.5} + 1) = 100 \ln(2.6487) = 97.41$$

$$\text{you pay } 97.41 - 69.31 = \$28.10 \quad (\text{avg } 56.2\text{¢/share}), \quad p_Y^{\text{new}} = \sigma(0.5) = 0.622$$

The price moved from 50¢ to 62.2¢ during your trade, and you paid an average in between.

- If YES: you collect \$50, a profit of +\$21.90. That matches $b \ln(0.622/0.5) = 100 \ln 1.245 \approx 21.9$ from §6.
- If NO: you lose \$28.10.

9. The hardware version

Write $d = q_Y - q_N$ and $\text{softplus}(x) = \ln(1 + e^x)$. From §4, $C = q_N + b \cdot \text{softplus}(d/b)$. So:

$$p_Y = \sigma\left(\frac{d}{b}\right) \quad \text{cost to buy } \Delta \text{ YES} = b \left[\text{softplus}\left(\frac{d + \Delta}{b}\right) - \text{softplus}\left(\frac{d}{b}\right) \right]$$

The entire market maker state is one integer, d . Both functions depend only on d , so a Python script precomputes them into two block-RAM lookup tables. Picking b as a power of 2 makes $\div b$ a bit shift. Each quote is then a counter, two lookups, and a subtract, rounded in the house's favor. The quoting layer (spread, inventory skew) and the risk limits sit on top of that core.